

Module 4 — Viva Speech (Visal)

Payments and Memberships. Roughly 5 minutes at a normal speaking pace. Square brackets are delivery notes, not words to say.

Opening — what I own (about 30 seconds)

Good morning. My name is Visal, and I was responsible for **Module 4 of HomeHero — Payments and Memberships.**

In simple terms, my module handles every point where money moves through the system. There are two of them. First, when a client pays a service provider for a completed job. And second, when a service provider pays HomeHero for their monthly membership.

Alongside that I own the review submission that finalises a booking, the provider's payment settings, and the earnings and revenue records that other modules report on.

The features I developed (about 1 minute)

[Count them off — don't rush this part, it's the list they're marking.]

I built **six** main features.

One — client job payment, with two paths: cash and card.

Two — the platform commission. HomeHero takes five percent on every card payment.

Three — provider earning records. Every card payment creates a record of exactly what that provider earned on that job.

Four — the review flow. Submitting the review is what actually completes a booking.

Five — the membership lifecycle — purchase, renewal, pricing, expiry, the grace period, and forcing a provider offline when they don't renew.

Six — provider payment settings, the bank details that appear as the payee when a client pays by card.

System flow — the job payment (about 1 minute 15)

Let me walk through the main flow.

Once a provider has finished a job, the client opens the payment page from their bookings. Before anything is shown, the backend checks four things: that the booking exists, that it belongs to *this* client, that it is in Accepted status, and that it has not already been paid.

The client then chooses cash or card.

If they choose cash, HomeHero is not involved in the money at all. So we record the payment method only. No amount, no commission, no earning record. The client pays the provider directly in person.

If they choose card, the provider's bank details are filled in automatically from their payment settings, and the client enters the agreed service amount. The system then charges the card.

[Slow down here — this is the part worth landing.]

Now, the important design decision. **The card is charged first, before anything is written to the database.** If the card is declined, we throw an error immediately and record nothing — there is no payment row, no revenue, no earning record. There is nothing to undo, because nothing was ever written.

If the charge succeeds, three records are written **inside a single database transaction**: the payment itself, HomeHero's five percent commission, and the provider's earning for that job. Because they are in one transaction, it is impossible to end up with a payment that has no revenue attached, or revenue with no payment.

Finally the client submits their review — and that is the step that moves the booking to Completed.

The five percent (about 30 seconds)

I want to highlight one thing about the commission.

The five percent is not calculated in JavaScript. It is a generated column in the PostgreSQL database. The database computes the platform fee and the total from the service amount, and stores them automatically.

This means the fee cannot be tampered with from the browser. Even if someone modified the frontend and sent a different total, the database would still calculate five percent from the service amount. My application code never writes those two values — it reads them back after the insert.

Membership lifecycle (about 45 seconds)

On the membership side, a provider pays LKR 4,999 per month for one service category. For two categories, the first payment is double — LKR 9,998 — and after that renewals are one and a half times, LKR 7,498.50.

The provider **never sends the price**. They only submit their card details. The backend works out what they owe from how many categories they offer and whether this is their first payment or a renewal.

When a membership expires, a scheduled job that runs every hour marks it expired and emails the provider. They then get a **three-day grace period** where they keep working normally. If they still have not renewed after three days, the same job forces them offline so they stop receiving new bookings. Renewing restores them immediately, in the same transaction as the payment.

Validation (about 45 seconds)

I validated at **three levels**.

First, the frontend — required fields, card number format, amount must be positive. This is only for the user's convenience; I never trust it.

Second, the backend. Every request goes through a schema before it reaches my code — card number must be sixteen digits, expiry must be MM slash YY, CVV three or four digits, rating between one and five.

But schema validation only checks the *shape* of the data. So there is a third layer: **business rules**. Is this booking yours? Is it in the right status? Has it already been paid? Has it already been reviewed? Is your membership still active, in which case you cannot renew yet?

And underneath all of that, the database enforces its own rules. One payment per booking, one review per booking, one earning record per payment, and one active membership per provider — all as unique constraints. So even if every layer above failed, the database would still refuse the duplicate.

Notifications (about 30 seconds)

My module sends three messages.

One in-app notification: when a client pays by card, the provider is notified with the booking number and the exact amount they received. Cash payments send nothing, because HomeHero never handled that money.

And two emails: one confirming a membership purchase or renewal, and one when a membership expires, telling the provider their grace-period deadline.

All three are sent **after** the database transaction commits, and the notification is wrapped in its own error handling — if sending fails, it is logged, but it can never undo a payment that has already succeeded.

Closing (about 15 seconds)

So in summary: my module handles both money flows in HomeHero, keeps HomeHero's revenue and the provider's earnings as separate records, calculates the commission in the database where it cannot be tampered with, and protects every write with transactions and constraints.

Thank you — I'm happy to take any questions.

60-second version

[If they cut you short.]

I was responsible for Payments and Memberships. That covers two money flows: clients paying providers for completed jobs, and providers paying HomeHero for their monthly membership.

For job payments, the client chooses cash or card. Cash is recorded only, since HomeHero isn't involved. For card, we charge first, and only if that succeeds do we write the payment, HomeHero's five percent commission, and the provider's earning — all in one transaction, so they can never drift apart. The five percent itself is a generated column in the database, so it can't be tampered with from the browser.

For memberships, the backend decides the price from the provider's category count. When one expires, an hourly job emails the provider and starts a three-day grace period, then forces them offline if they still haven't renewed.

Everything is validated in the frontend, again in the backend schema, again as business rules, and finally by unique constraints in the database.